

LAB1- RVC

Goals of the Lab

- Requirement engineering
 - Analysis
 - Formalization (context diagram, FR, ..)
 - Consistency

High level, informal description

- Since several years robotic vacuum cleaners (RVC) are available. An RVC is capable of cleaning the floors of a house in autonomous mode.
- An RVC system is composed of the robot itself and a charging station. The charging station is connected to an electric socket in the house, and allows charging the battery on board of the robot.
- The robot itself is composed of mechanical and electric parts, a computer, and sensors. One infrared sensor in the frontal part recognizes obstacles, another infrared sensor always on the frontal part recognizes gaps (like a downhill staircase). A sensor on the battery reads the charge of the battery. The computer collects data from the sensors and controls the movement of four wheels. Another sensor on one of the wheels computes direction and distance traveled by the robot.
- Finally on top of the robot there are three switches: on-off, start, learn.
- The learn button starts a procedure that allows the robot to map the space in the house. With a certain algorithm the robot moves in all directions, until it finds obstacles or gaps, and builds an internal map of this space. By definition the robot cannot move beyond obstacles, like walls or closed doors, and beyond gaps taller than 1cm.
- The starting point of the learn procedure must be the charging station. When the map is built the robot returns to the charging station and stops.
- The start button starts a cleaning procedure. The robot, starting from the charging station, covers and cleans all the space in the house, as mapped in the 'learn' procedure.
- In all cases when the charge of the battery is below a certain threshold, the robot returns to the charging station. When recharged, the robot completes the mission, then returns to the charging station and stops.
- The robot has also a USB port to be used by maintenance staff for diagnostics and possibly firmware update

4 documents

- Requirements, system level
 - Requirements robot
 - Requirements firmware
 - Requirements charging station

System Requirements document

- Business model
- Stakeholders
- Context diagram
- Functional requirements
- NF requirements
- Table of access rights
- Use cases / scenarios
- Conceptual diagram / Glossary
- System design
- Deployment diagram

The core functions

- Clean house
 - Learn space
 - Move autonomously

Business model

- Answers those questions
 - Who? (customer segments)
 - Home segment (no industrial cleaning)
 - Basic model (no app, no internet connection, no alexa..)
 - Why? (value proposition)
 - Save effort and time to clean house
 - Money flow? (revenue stream)
 - From users / customers

Context

- Company X developing consumer devices willing to enter market (OEM)
 - Termozeta, begHELLi
 - Philips, Siemens, hoover
 - dyson
- Marketing office
- Engineering office (R&D)
 - Software office
 - Electronics and mechanical office

Business model

- See

Business Model Canvas



Stakeholders

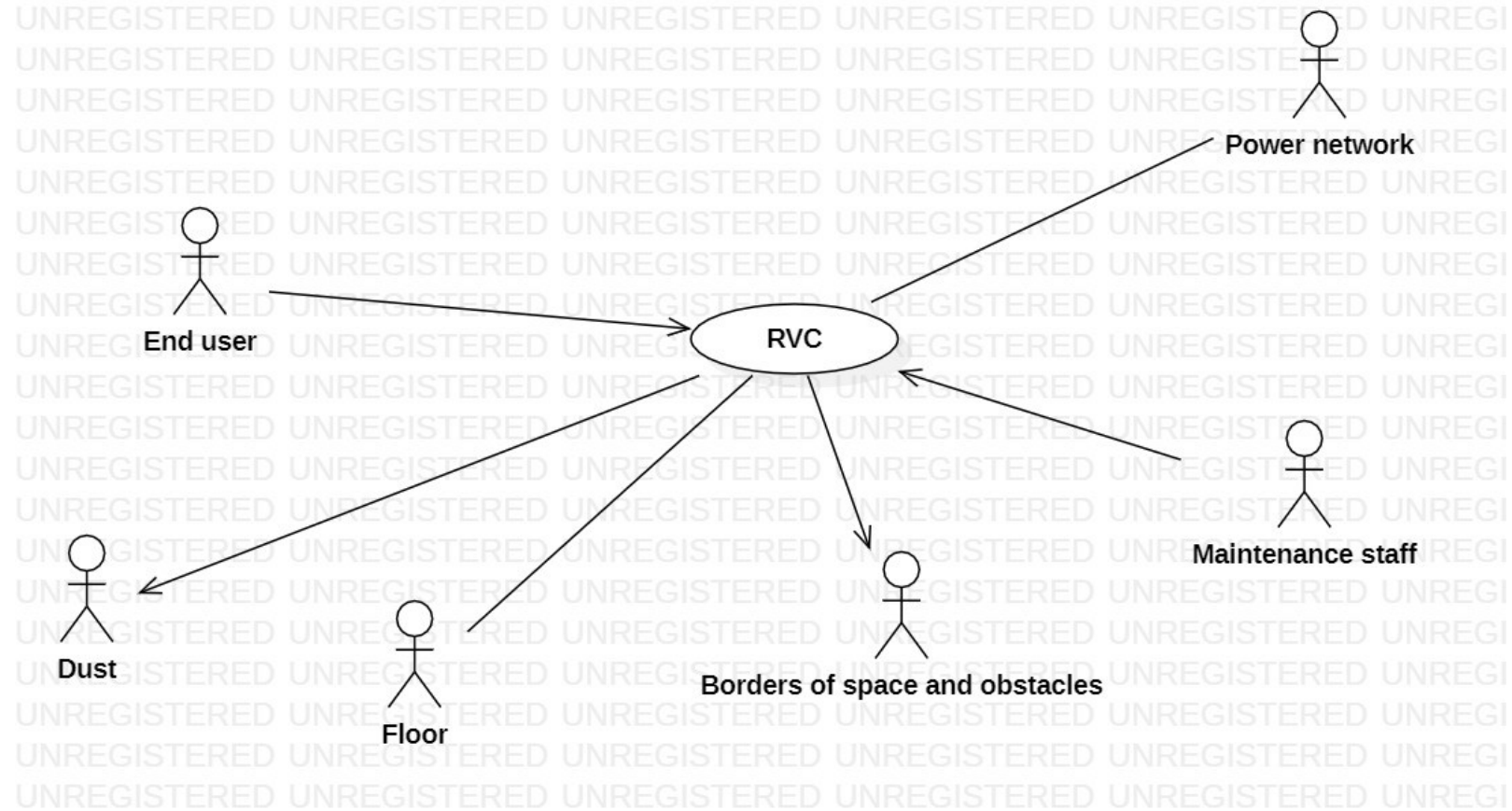
- End user
 - Marketing office
 - (should know requirements of end users, and define FR NFR for R&D)
 - R&D office
 - Sw developers
 - All suppliers of components used (tier1)
 - Hardware suppliers (memory, cpu, sensors..)
 - Floor
 - Dust
 - Borders and obstacles
-
- Maintenance staff

----- system requirements



Requirements, system level
Requirements robot
Requirements firmware
Requirements charging station

Context diagram



Interfaces

Actors	Physical interface	Logical interface
End user	3 buttons display	Start , learn, clean Various signals (malfunctions)
Power network	Schuko plug	220v, 50hz
Borders of space and obstacles	IR sensor for obstacle, IR sensor for gap	Recognize obstacle, recognize gap
dust	Air flow	Vacuum generation
floor	wheels	Move
Maintenance staff	Usb port type A	diagnostic Firmware upgrade

System design

- Robot
- Charging station

FR

Sys-FR1	Manage movement
Sys-FR1.1	go to charging station (go to position (x,y)
	go straight
Sys-Fr1.2	turn
1.3	stop
1.4	recognize obstacle
1.5	
	recognize gap
	compute current position
FR2	Monitor energy
Sys-FR2.1	Charge battery
2.2	Read energy capacity
2.3	
	Switch on / off

FR

FR3	Communicate with user
FR	Signal full (of dust)
	Signal stuck
Fr	Signal energy low
1.3	
1.4	
1.5	
Sys-FR4	Maintenance
	Diagnose system
	upload new firmware
SysFR5	Clean floor

Table of rights

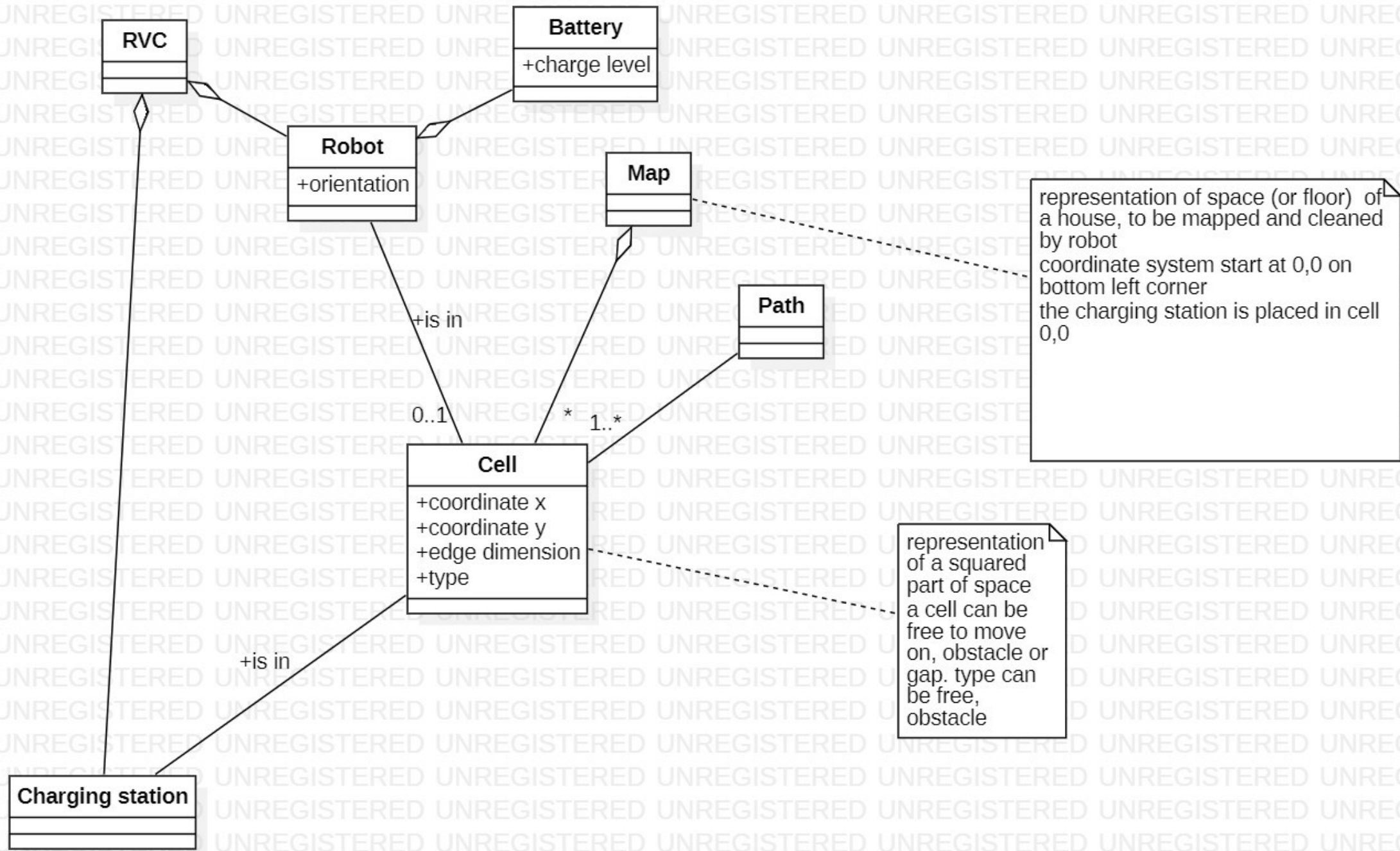
	End user	Maintenance staff			
FR1	Y	Y		N	N
FR2	Y	Y		N	N
FR3	Y	Y			
Sys-FR4	N	Y			
FR5					
FR6					

NFR

NFR	Measure
Usability	Any person > 14 yrs old should be able to use system as follows: -first usage: have robot start mapping in < 5 minutes – first usage after mapping: have robot start cleaning in < 1 minute
	weight <5kg
	Any person > 14 yrs old should be able to change dust bag in < 2 min
Cost	System should cost < ?? euro
Efficiency	House up to 80sqm should be cleaned in < 1hr Mapping of space, first time, 80 sqm, should take < ?? minutes
	Should clean efficiently (test: putting 10g of dust on 1sqmeter, after robot passes only 1 g on floor and 9g in dust bag)
safety	No harm to people, animals, things in the house
Reliability	Work for at least 1000 hrs before any component fails
Portability	No (does not apply to system)
maintainability	Possibility to change dust bag
security	No un authorized person should be able to control robot

Glossary

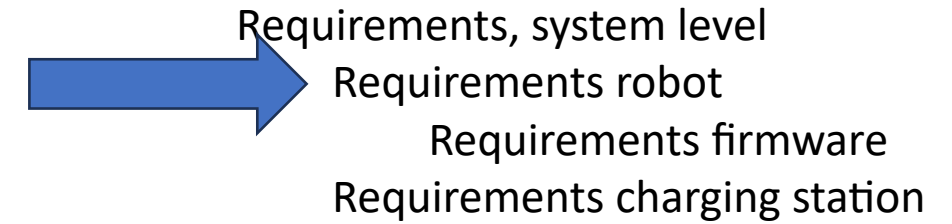
- To be clarified:
 - House, Map, space, floor, position, obstacle, gap



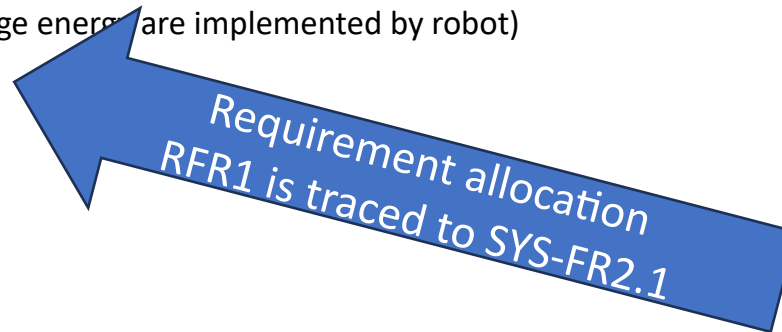
Deployment diagram

- Non useful here

---- robot requirements



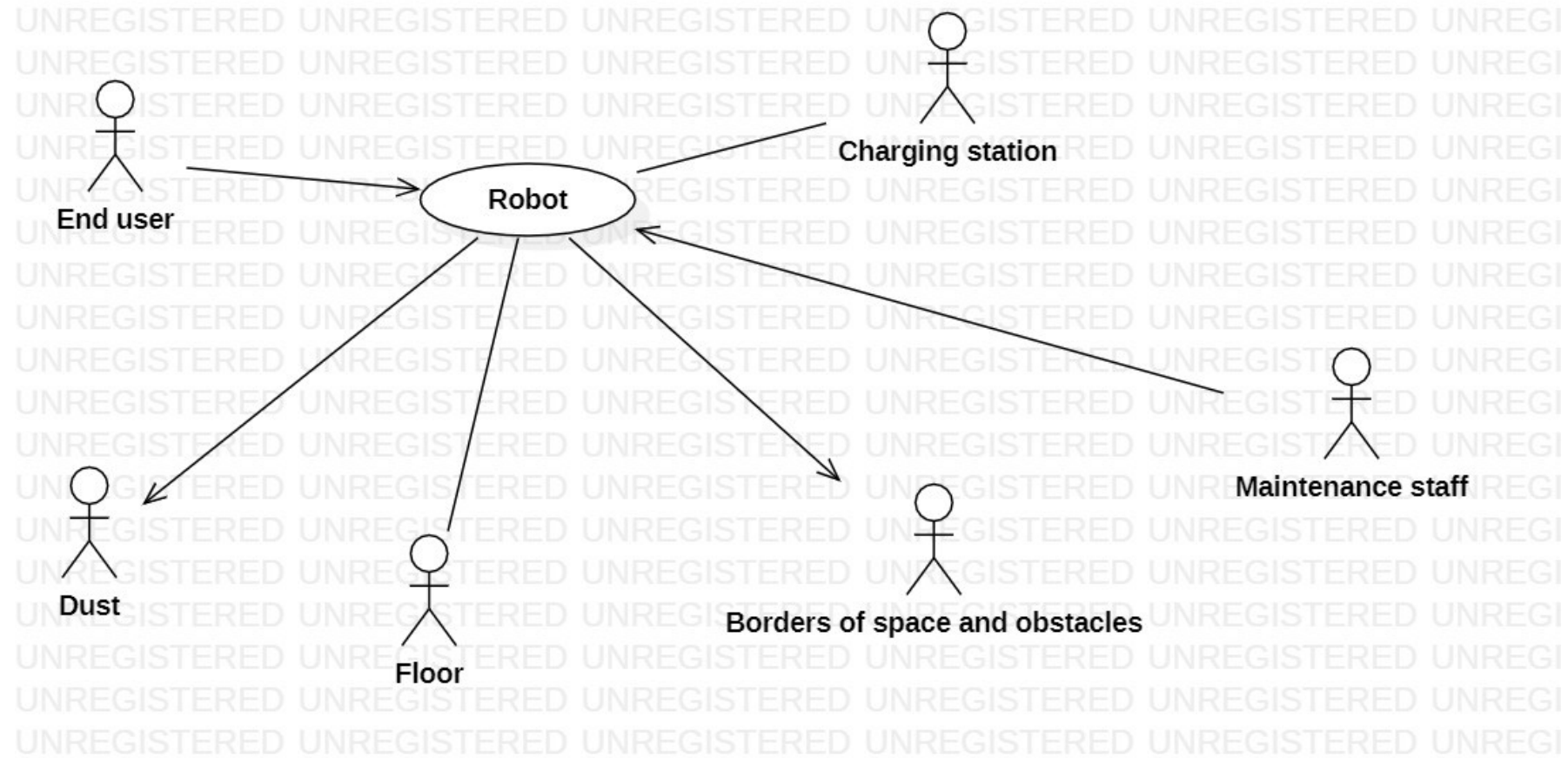
- Context diagram
- Interfaces
- Fr
 - R-FR1 Read battery level
 - R-FR2 Manage movement
 - (TBD, nearly all requirements except part of manage energy are implemented by robot)
- Nfr
- System design
 - 4 wheels
 - 2 engines for moving 2 wheels
 - 2 engines for rotating 2 wheels
 - Ir sensor obstacles
 - Ir sensor gaps
 - Battery
 - Battery Charge sensor
 - Computer and firmware
 - Dust bag
 - Sensor dust bag
 - 3 buttons
 - Led for malfunctions
 - Vacuum pump



NFR

- Usability: same as for system
- Cost: same as system minus cost of charging station
- Efficiency: same as system
 - Translate to: battery at least ??maH
 - Consumption of engines...
- Safety : same as system
 -

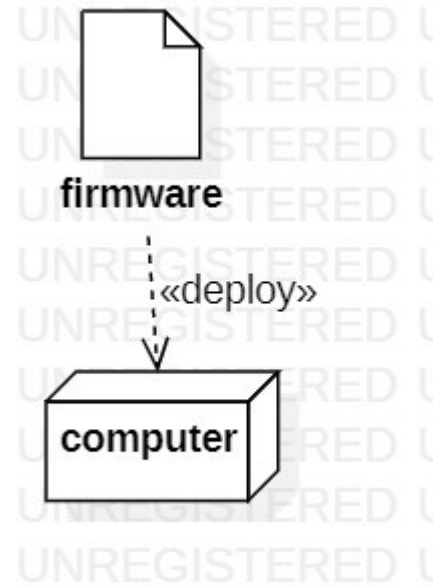
Context diagram



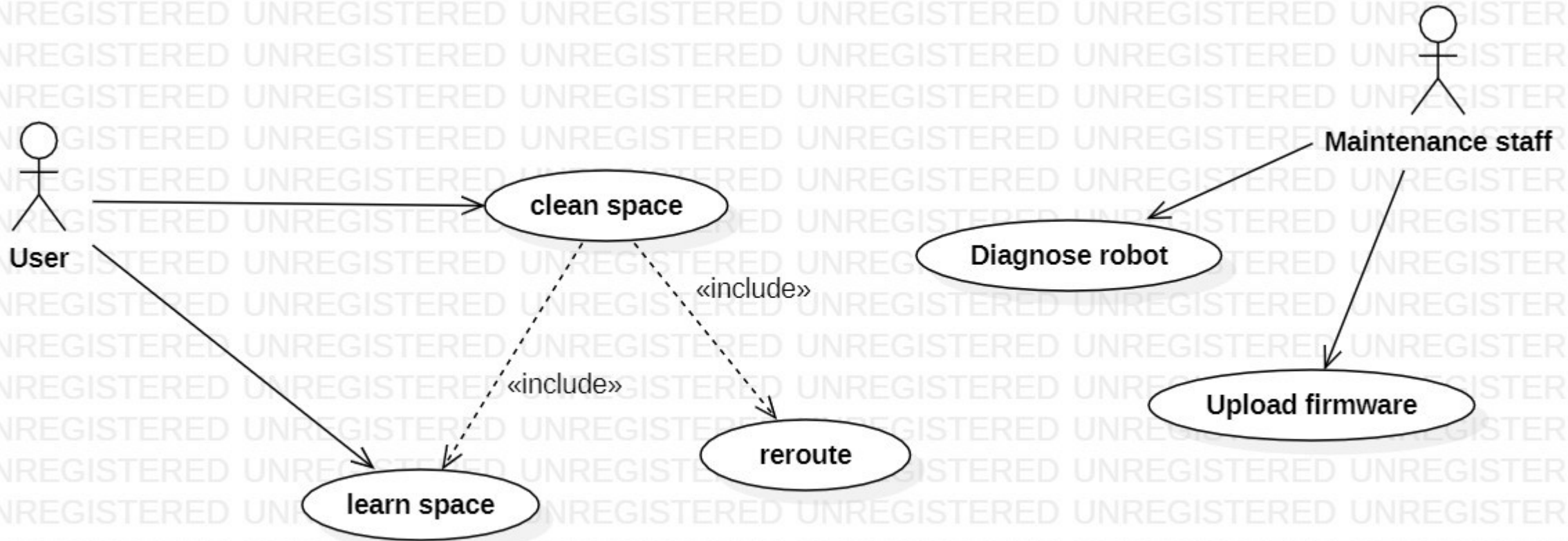
Interfaces

Actors	Physical interface	Logical interface
End user	3 buttons	Start , learn, clean
Charging station	Two pole connector (to be further defined)	20V dc, max X amp
Borders of space and obstacles	IR sensor for obstacle, IR sensor for gap	Recognize obstacle, recognize gap
dust	Air flow	Generate Vacuum
floor	Wheels	Move, rotate
Maintenance staff	Usb port type A	diagnostic Firmware upgrade

Deployment diagram



Use cases and scenarios



UC1 Clean space

- Actor involved: end user
- Goal: clean the space
- Description: robot moves from charging station, following a path capable of covering the whole space, avoiding obstacle. In each cell the robot cleans dust, then proceeds to next cell. Finally robot returns to charging station
- Scenarios: s1, s1V, s1-exc1, ..
- exceptions: battery not enough

- S1 (all right, no obstacles)
 - Precondition: robot is in charging station, battery is full, robot is working, map of space is available
 - Post condition: all cells of space are clean, robot is in charging station
- End user pushes clean button
- Robot starts diagnose procedure, result is ok
- Robot computes path to cover all space (FW-FR1.3) (ex, path is cA1, cA2, c B2,..)
- Robot starts vacuum pump
- Robot moves to Ca1, stays enough time to clean it
- Robot computes and updates its position and orientation
- Robot moves to cA2,
- Robot computes and updates its position and orientation
- ...
- Robot back to charging station

- S1V (all right, one obstacle found)
 - Precondition: robot is in charging station, battery is full, robot is working, map of space is available
 - Post condition: all cells of space are clean, robot is in charging station
- End user pushes clean button
- Robot starts diagnose procedure, result is ok
- Robot computes path to cover all space (FW-FR1.3) (ex, path is cA1, cA2, c B2, cB3)
- Robot starts vacuum pump
- Robot moves to Ca1, stays enough time to clean it
- Robot computes and updates its position and orientation
- Robot moves to cA2,
- Robot computes and updates its position and orientation
- Robot recognizes obstacle in cell cB2 that was not in map. Robot computes turn around of cell cB2, and continues to cB3
- ...
- Robot back to charging station

- S1-exc1
 - Precondition: robot is in charging station, battery is full, robot is working, map of space is NOT available
 - End user pushes clean button
 - Robot starts diagnose procedure, result is ok
 - Robot does not move, and signals on display to do 'Learn'

- S1-exc2
 - Precondition: robot is in NOT charging station, battery is full, robot is working, map of space is available
 - End user pushes clean button
 - Robot starts diagnose procedure, result is ok
 - Robot does not move, and signals to put it in charging station

- S1-exc3
 - Precondition: robot is in charging station, battery is NOT full, robot is working, map of space is available
 - End user pushes clean button
 - Robot starts diagnose procedure, result is ok
 - Robot does not move, and signals on display to do wait to finish charging

- S1-exc4
 - Precondition: robot is in charging station, battery is full, robot is NOT working, map of space is available
 - End user pushes clean button
 - Robot starts diagnose procedure, result is NOK
 - Robot does not move, and signals 'broken, call maintenance'

- S1exc5 (someone moves robot in middle of mission)
 - Precondition: robot is in charging station, battery is full, robot is working, map of space is available
 - Post condition: robot stops and waits for user intervention
- End user pushes clean button
- Robot starts diagnose procedure, result is ok
- Robot computes path to cover all space (FW-FR1.3) (ex, path is cA1, cA2, c B2,..)
- Robot starts vacuum pump
- Robot moves to Ca1, stays enough time to clean it
- Robot computes and updates its position and orientation
- Robot moves to cA2,
- Robot computes and updates its position and orientation
- ---- someone moves robot
- Robot recognizes map is wrong
- Robot stops and signals on display: 'bring back to charging station'

- S1exc6 (not enough battery)
 - Precondition: robot is in charging station, battery is full, robot is working, map of space is available
 - Post condition: space not fully cleaned, robot in charging station
- End user pushes clean button
- Robot starts diagnose procedure, result is ok
- Robot computes path to cover all space (FW-FR1.3) (ex, path is cA1, cA2, c B2,..)
- Robot starts vacuum pump
- Robot moves to Ca1, stays enough time to clean it
- Robot computes and updates its position and orientation
- Robot moves to cA2,
- Robot recognizes not enough charge to finish cleaning
- Robot stops vacuum pump
- Robot computes path P-BACK to charging station
- Robot follows P-BACK

UC2 Map space

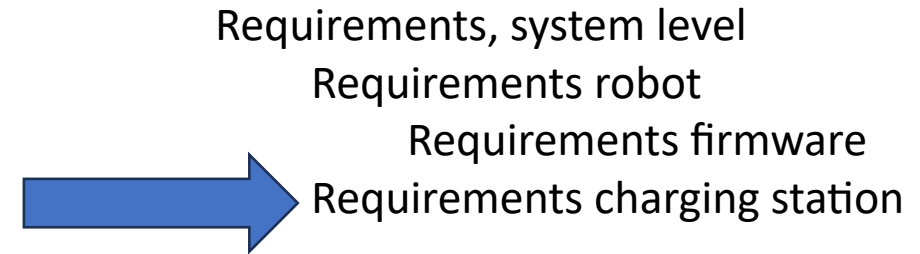
- Actor involved: end user
- Goal: map the space
- Description: robot moves from charging station, applying an algorithm to cover a graph so that all cells are tagged as free or obstacle or gap. In the end the space is described by a map
- Scenarios: s2
 - exceptions: battery not enough, robot breaks, robot not in charging station, map already exists

- S2 (all right, map produced)
 - Precondition: robot is in charging station, battery is full, robot is working, map does not exist
 - Post condition: map is produced (every cell in space is tagged free, obstacle or gap), robot is in charging station
- End user pushes learn button
- Robot starts diagnose procedure, result is ok
- Robot moves to cell xy applying coverage algorithm
- Cell xy is free (no obstacle found), cell xy is tagged as free in map
- Robot moves to cell kw applying coverage algorithm
- Cell kw is not reachable, cell is tagged as obstacle
- ..
- ...(until all cells tried and tagged)
- Robot back to charging station

- S2-exc (map already available)
 - Precondition: robot is in charging station, battery is full, robot is working, map **exists** already
 - Post condition:
 - End user pushes learn button
 - Robot asks 'are you sure?'
 - End user does nothing
 - Robot does not move

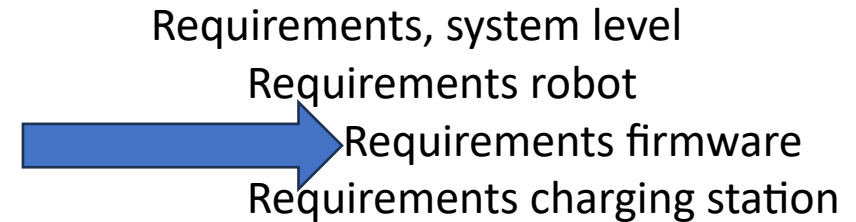
---- charging station requirements

- Context diagram
- Interfaces
- Fr
 - Charge battery
- Nfr
- System design
 - Transformer 220v ac to 20v dc
 - ..



---- firmware requirements

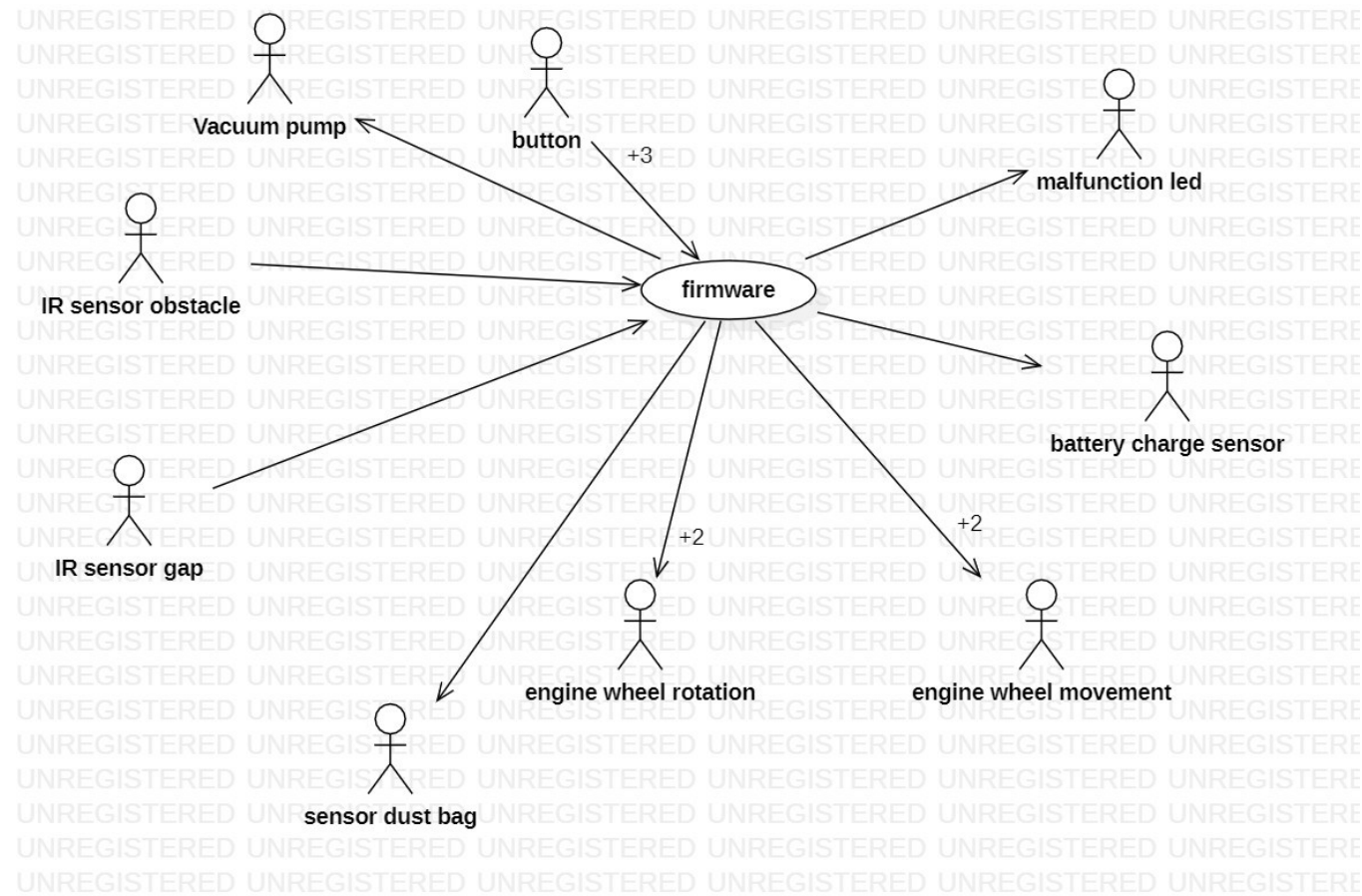
- Context diagram
- Interfaces
- Fr
 - (nearly all robot-fr will be partially or fully allocated here. Ex 'movement' is allocated to wheels and wheel engines for actual movement, and to firmware for controlling movements)
- Nfr
- Glossary
- (software design)



FR

ID	description	Traced to
FW-FR1	Manage movement	R-FR2
	Compute path to charging station Compute energy needed for covering path (path) Compute time needed for covering path (path)	
	Build map	
FW-FR1.3	computeOptimalPath to cover all map and return to charging station	
FW-FR1.5	followPath Reroute in case of obstacle (recompute path) Compute robot position and orientation Recognize that map is wrong (space does not match with map, ex someone has moved the robot)	
	moveRobotforward(distance) moveRobotBackward(distance) Stop	
	turnRobotRight(degrees) // controls rotation and movement of two wheels to turn robot on the spot turnRobotLeft(degrees)	
	Manage Sensors	
	Read battery capacity Read max battery capacity Read battery error condition	
	Move wheel (id wheel, degrees)	

Context diagram

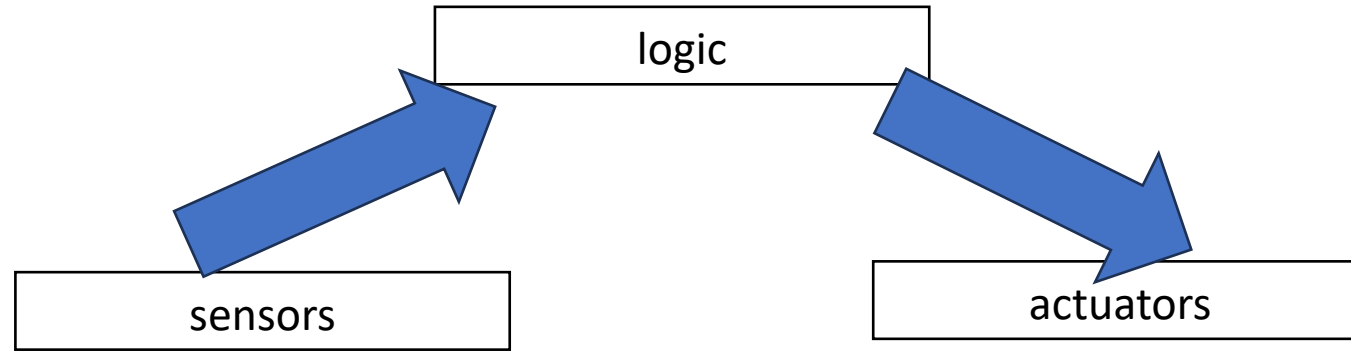


Interfaces

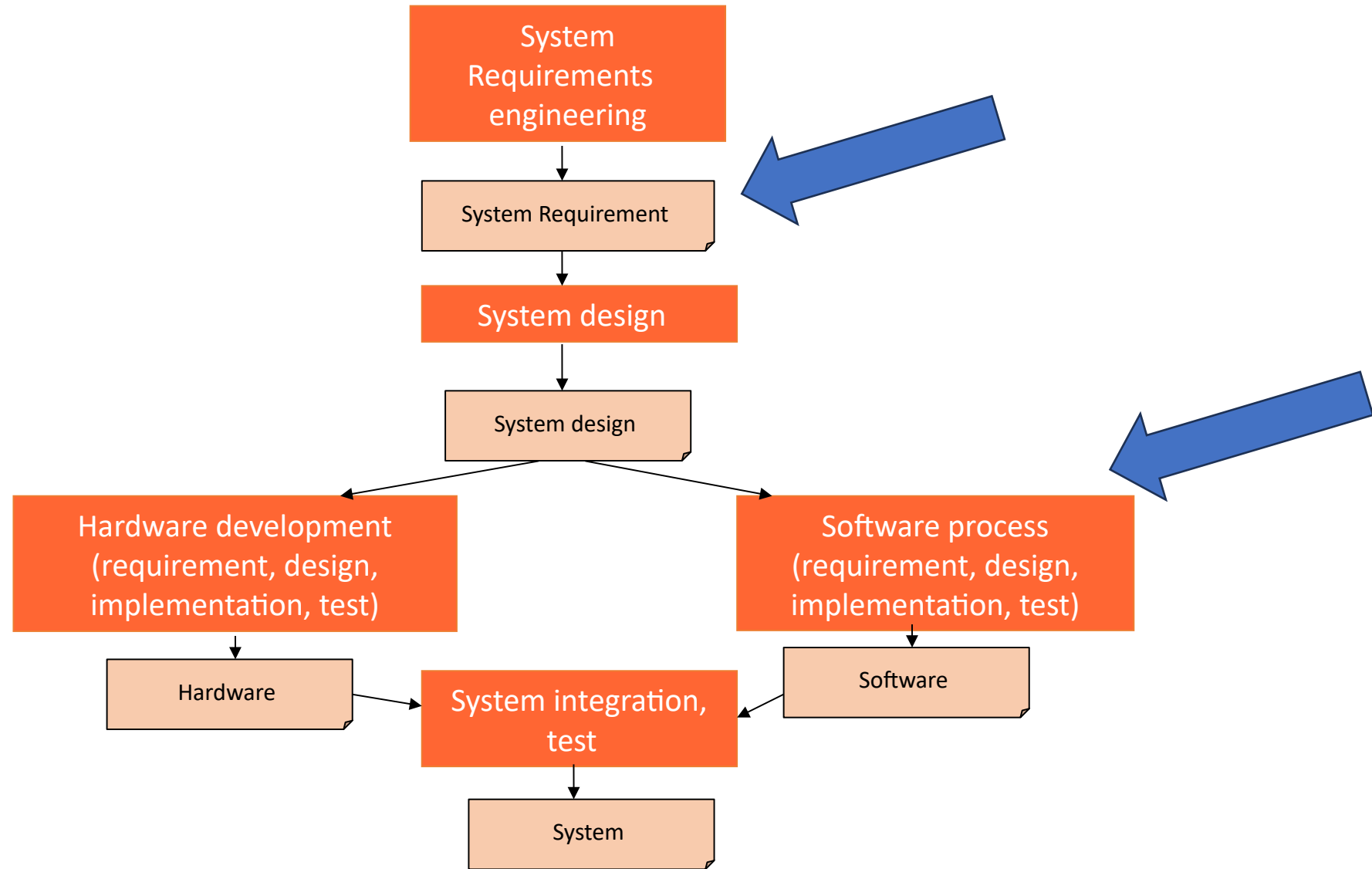
Actors	Physical interface	Logical interface
Button on off		Variable Y, true false Start , stop
Button learn		
Ir obstacle sensor		Variable Z,
Battery sensor		Variable B, long int 0 = 0 charge Maxint = 1Ah Variable Berror, short int 1 =error1 2 = error2 Variable MaxC..

- Interfaces to be further defined, depending on the actor (component)
- Ex the obstacle sensor could be either connected to an interrupt function (interrupt, asynchronous model), or via a memory variable to be read regularly (polling model)

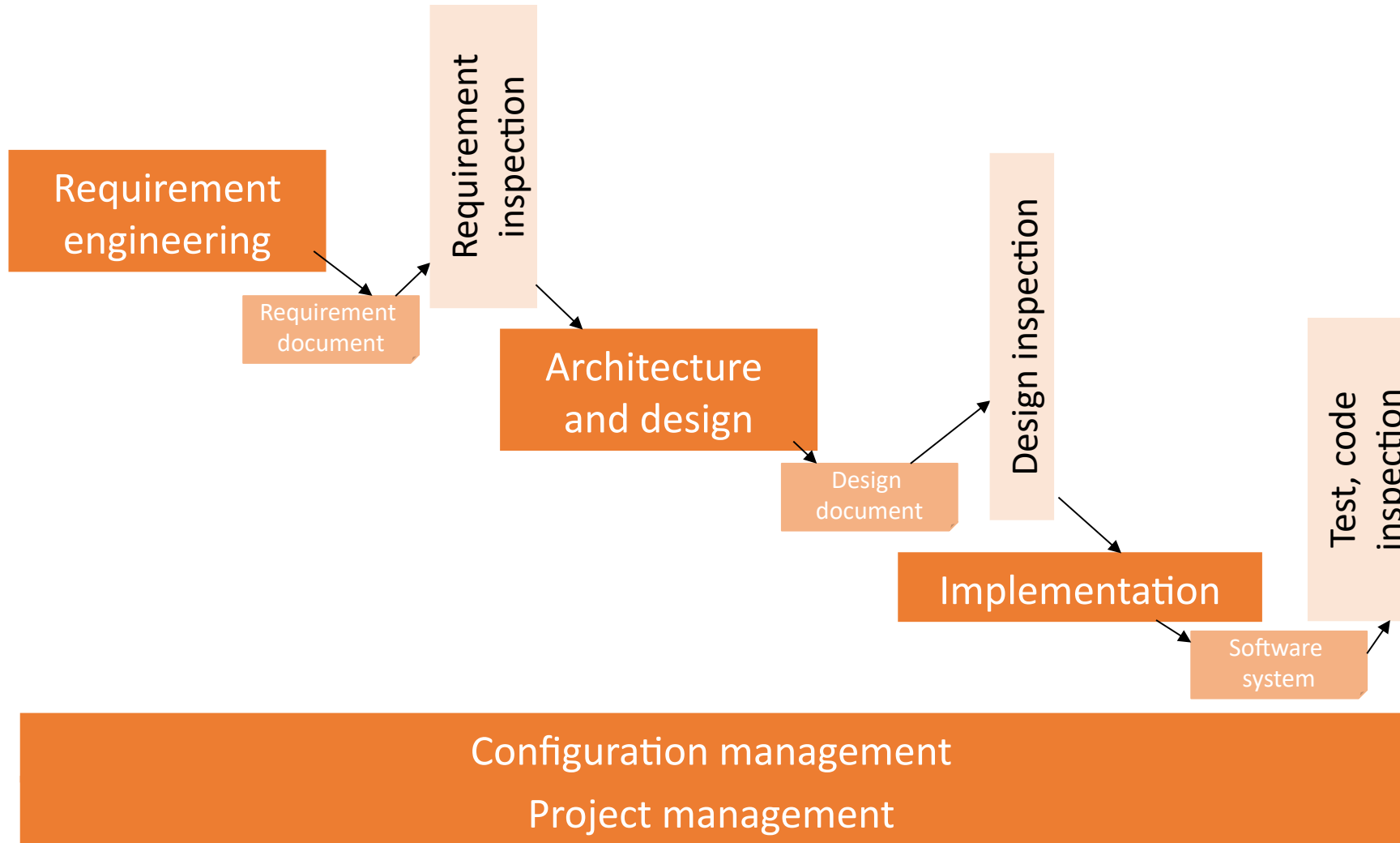
Embedded system



System process



Software process



System vs sw requirements

System requirements

- Functional

- F1
- F2
- F3

Software requirements

- Functional

- SWF1
- ~~SWF2~~
- SWF3

- Non functional

- NFR1
- NFR2

- Non functional

- SWNFR1
- ~~SwNFR2~~

System vs software requirement

- System context diagram
- Software context diagram

Design

- The following refers to the software design of firmware onboard of robot (slides 41-44)

Architectural choices

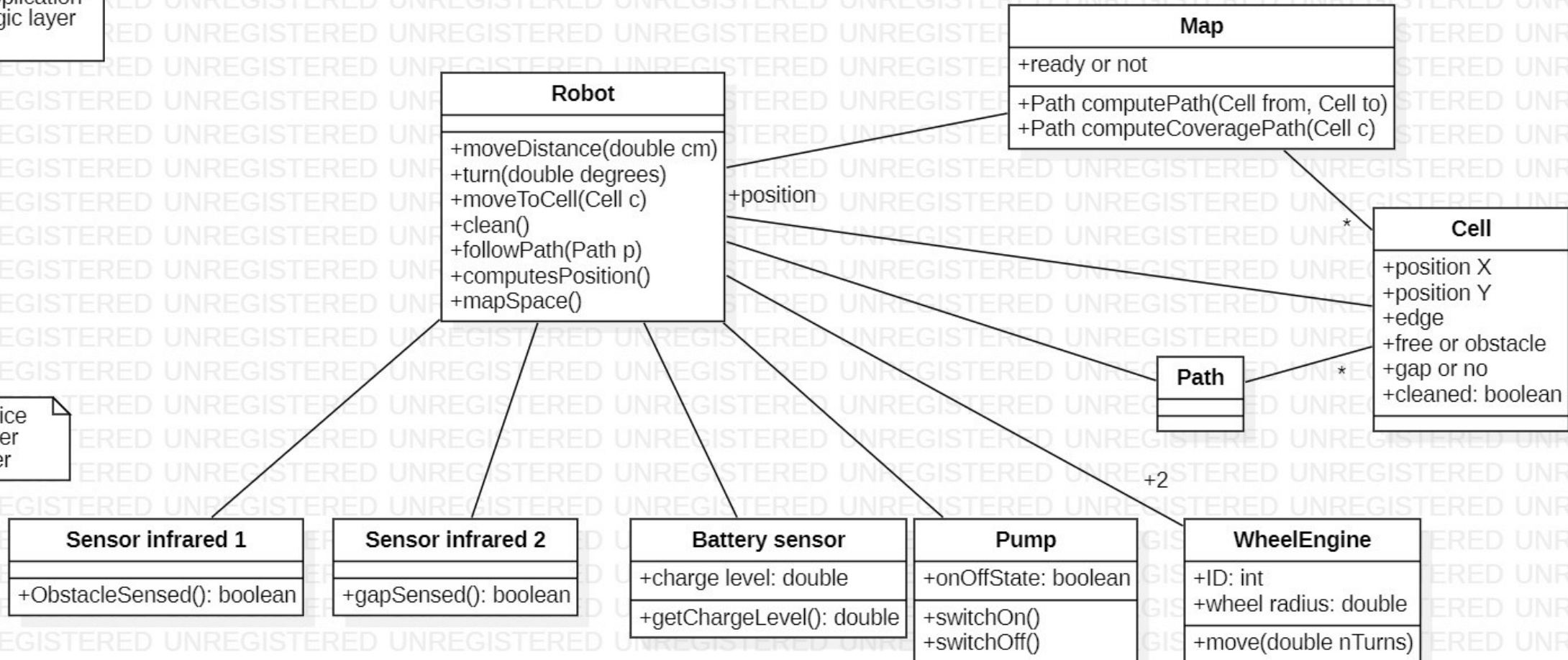
- Layered pattern
 - Maintainability
 - (ease of changing layers independently)
 - Especially device driver layer
- Communication model
 - One process (instead of many threads or processes)
 - Procedure calls (because one process)
 - Polling (instead of interrupts)
 - (simplicity)

Design

Interaction layer

application logic layer

device driver layer



- Design patterns used

- Proxy (in data layer, access to device drivers of sensors and actuators is NOT made directly (especially by classes in business logic layer) but using proxy class (advantage: maintainability)
- Adapter (in proxy class, methods translate from local protocol to device driver protocol) example: Pump.switchOn() defines internal protocol, and translates to device driver protocol (ex set certain interface variable to 1).

Robot class

- Has reference to Map m and Cell position
- Void move(double cm)
 - Move the robot straight (no turning) for the given cms
 - Computes the number of turn per right and left wheel
 - Calls move(nturns) in right and left wheel
 - Stops if there is an obstacle (catch exception from obstacle sensor)
 - Calls computePosition()
- Void turn(double degrees)
 - Compute the number of turns (in opposite directions) per right and left wheel to obtain the given rotation in degrees
 - Calls move(nturns) in right and left wheel
 - (robot does not change position but only rotates, so no need to update position or check obstacles (gaps))
- moveToCell (Cell c)
 - Path p = computePath(position, c)
 - If path can be computed foreach {Cell c in p, compute distance d in cm to c, compute needed rotation r, call rotate(r), call move(d) }
 - Else ???
- computePosition()
 - Change position of robot using number of rotations of wheels and circumference of wheels

Robot class

- Clean()
 - If map is !ready return error
 - If robot is not in charging station return error
 - If battery is not fully charged return error
 - Path p= m.computeCoveragePath(position) // position must be
 - //charging station
 - pump.switchOn()
 - followPAth(p); // exceptions? Like battery down, obstacles??
 - pump.switchOff()
 -
- FollowPath (Path p)
 - foreach {Cell c in p, compute distance d in cm to c, compute needed rotation r,
call rotate(r), call move(d) }
- MapSpace()
 - Starting from charging station explore all space, find empty cells or obstacles or gaps, build a corresponding map

Path class

- Defines a sequence of Cells (ex List of Cells)

Map class

- Defines a matrix of cells (ex Array 2D of Cells)
- Path computePath(Cell from, Cell to)
 - Using the map with obstacles and gaps computes a path from, to
 - Returns true if path can be computed
 - Returns false otherwise
- Path computeCoveragePath(Cell c)
 - Using the map computes path from c to c that covers ALL free cells in the map possibly never passing more than once on same cell